

High Concurrency Ledger System

1. Introduction

1.1 Purpose

This SRS defines requirements for a distributed transaction engine that supports horizontal scalability through sharding, strong consistency through atomic transactions, and fault tolerance through replication. The system is inspired by Google Spanner and CockroachDB but simplified for educational purposes.

1.2 Scope

The **Distributed Sharded Transaction Engine** enables atomic money transfers across distributed shards with ACID-like guarantees.

Key Features:

- Horizontal data partitioning across multiple shards
- Atomic cross-shard transfers using two-phase commit (2PC)
- Fault tolerance via primary-follower replication
- Durability through write-ahead logging
- Automatic failure detection and recovery

1.3 System Overview

The system consists of the following components (per UML diagram):

- **APIService**: Receives and validates transaction requests
- **TransactionCoordinator**: Orchestrates 2PC protocol
- **ShardManager**: Maps accounts to shards
- **Shard**: Contains replicas managing the account subset
- **Replica**: Maintains ledger and WAL (PRIMARY or FOLLOWER role)
- **Ledger**: Stores account balances
- **WriteAheadLog**: Records operations before execution
- **KafkaLog**: Distributed transaction log
- **ReplicaManager**: Handles replica lifecycle and failover
- **FailureDetector**: Monitors replica health via heartbeats

1.3 Definitions and Acronyms

Term	Definition
Shard	Horizontal partition of data containing subset of accounts
Replica	Copy of shard data (PRIMARY or FOLLOWER)
WAL	Write-Ahead Log ensuring durability
2PC	Two-Phase Commit protocol for atomic distributed transactions
Coordinator	Component orchestrating cross-shard transactions

2. Functional Requirements

2.1 Transaction Processing

REQ-TX-001: Transaction Submission

- System SHALL accept transactions with: transaction ID (UUID), source account, destination account, and amount
- APIService SHALL validate: positive amount, distinct accounts, valid UUIDs
- Invalid transactions SHALL be rejected with appropriate error codes

REQ-TX-002: Single-Shard Transactions

- For transactions where both accounts reside on same shard, system SHALL:
 - Execute locally without 2PC overhead
 - Write operations to WAL before applying to ledger
 - Update both account balances atomically
 - Replicate to follower replicas

REQ-TX-003: Cross-Shard Transactions

- For transactions spanning multiple shards, TransactionCoordinator SHALL execute 2PC:
 - **Prepare Phase:** Send PREPARE to all involved shards, wait for PREPARED/ABORT responses
 - **Commit Phase:** If all shards are prepared, send COMMIT; otherwise, send ABORT
 - Publish final decision to KafkaLog for durability
- Cross-shard transactions SHALL be atomic (all shards commit or all abort)

REQ-TX-004: Transaction Status

- System SHALL track transaction states: PENDING, PREPARED, COMMITTED, ABORTED
- Status transitions: PENDING → PREPARED → COMMITTED, or PENDING/PREPARED → ABORTED

2.2 Data Management

REQ-DATA-001: Sharding

- ShardManager SHALL deterministically map accounts to shards using hash-based or modulo partitioning
- Same account SHALL always map to the same shard
- System SHALL support a configurable number of shards

REQ-DATA-002: Account Ledger

- Each Replica SHALL maintain a Ledger storing account balances (Map<String, double>)
- Ledger SHALL provide atomic operations:
 - `debit(accountId, amount)`: Verify sufficient balance, subtract amount
 - `credit(accountId, amount)`: Add amount to balance
- Account balances SHALL never be negative

REQ-DATA-003: Write-Ahead Logging

- Each Replica SHALL append operations to WAL before applying to Ledger
- WAL entries SHALL contain: log ID, transaction ID, operation type, timestamp
- WAL SHALL synchronously flush to disk (fsync) before the operation completes
- WAL SHALL support replay for crash recovery

2.3 Replication and Fault Tolerance

REQ-REP-001: Primary-Follower Architecture

- Each Shard SHALL have one PRIMARY replica and multiple FOLLOWER replicas
- PRIMARY handles all write operations
- FOLLOWERS replicate PRIMARY's WAL entries

REQ-REP-002: Replication Protocol

- PRIMARY SHALL replicate log entries to FOLLOWERS
- PRIMARY SHALL wait for acknowledgment from the majority of replicas before committing
- FOLLOWERS SHALL apply log entries to the local ledger in order

REQ-REP-003: Failure Detection

- FailureDetector SHALL send periodic heartbeats to all replicas (default: 5-second interval)
- Replica SHALL be marked failed after missing multiple consecutive heartbeats (default: 3 misses)
- FailureDetector SHALL notify ReplicaManager of failures

REQ-REP-004: Primary Failover

- When PRIMARY fails, ReplicaManager SHALL:
 - Select the most up-to-date FOLLOWER
 - Promote FOLLOWER to PRIMARY role
 - Update ShardManager's shard registry
 - Notify the system of the new primary
- New PRIMARY SHALL replay WAL to ensure a consistent state

2.4 Distributed Coordination

REQ-COORD-001: KafkaLog

- KafkaLog SHALL organize entries into topics (transactions, per-shard replication)
- SHALL assign sequential offsets to entries
- SHALL guarantee total ordering within topics
- SHALL persist entries durably before returning

REQ-COORD-002: Transaction Recovery

- After the coordinator crash, the new coordinator SHALL:
 - Read transaction log from KafkaLog
 - Identify incomplete transactions
 - Contact participating shards to determine status
 - Complete 2PC (commit or abort) for in-flight transactions
-

3. Non-Functional Requirements

3.1 Performance

REQ-PERF-001: System SHALL support a minimum of 100 transactions/second per shard

REQ-PERF-002: Single-shard transaction latency < 50ms (p99)

REQ-PERF-003: Cross-shard transaction latency < 200ms (p99)

REQ-PERF-004: System SHALL scale horizontally with additional shards

3.2 Reliability and Safety

REQ-SAFE-001: System SHALL maintain invariants:

- Total money conserved (sum of balances constant)
- No negative balances
- All committed transactions are durable

REQ-SAFE-002: System SHALL tolerate:

- Loss of any single replica without data loss
- Primary failure with automatic failover
- Process crashes with full recovery via WAL replay

REQ-SAFE-003: Committed transactions SHALL survive single node crashes

3.3 Quality Attributes

REQ-QUAL-001: Code SHALL be well-documented with inline comments explaining design decisions

REQ-QUAL-002: System SHALL include comprehensive tests:

- Unit tests for each component
- Integration tests for transaction flows
- Fault injection tests for failure scenarios
- Correctness tests demonstrating atomicity

REQ-QUAL-003: As an educational system, code SHALL prioritize readability over micro-optimizations

4. System Architecture

4.1 Component Interactions

Based on the UML class diagram:

Transaction Flow (Cross-Shard):

1. APIService receives transaction, validates, and forwards to TransactionCoordinator
2. TransactionCoordinator queries ShardManager to identify involved shards
3. The coordinator sends PREPARE to each shard
4. Each Shard validates the transaction, writes to WAL, and responds PREPARED or ABORT
5. If all PREPARED: Coordinator publishes COMMIT to KafkaLog, sends COMMIT to shards
6. If any ABORT: Coordinator sends ABORT to all shards
7. Shards apply the transaction to the Ledger and replicate to followers

Failure Handling Flow:

1. FailureDetector monitors replicas via heartbeats
2. On failure detection, notifies ReplicaManager
3. ReplicaManager promotes the follower to the primary
4. New primary replays WAL to recover state

4.2 Data Flow

- **Write Path:** API → Coordinator → Shard → WAL → Ledger → Followers
- **Read Path:** Query → ShardManager → Shard → Ledger
- **Recovery Path:** Coordinator → KafkaLog → Transaction Resolution → Shards

4.3 Component Responsibilities

Component	Key Responsibilities
APIService	Request validation, UUID generation
Transaction Coordinator	2PC orchestration, transaction state management
ShardManager	Account-to-shard mapping, transaction routing
Shard	Local transaction execution, replica coordination
Replica	State management (Ledger + WAL), log application
WriteAheadLog	Operation persistence, replay support
KafkaLog	Distributed transaction log, ordered event stream
ReplicaManager	Failover, replica promotion, synchronization
FailureDetector	Health monitoring, failure notification

5. Constraints and Assumptions

5.1 Technical Constraints

- Must run on commodity hardware
- Use standard TCP/IP networking
- Implement in Go
- Use filesystem for persistence (no specialized storage)

5.2 Design Constraints

- Prioritize clarity and correctness over performance
- Follow industry patterns (Spanner, CockroachDB)
- Maintain clear component separation
- Minimize external dependencies

5.3 Assumptions

- Network is asynchronous but eventually reliable
 - Node failures are fail-stop (no Byzantine faults)
 - Disk storage is reliable (crash-recovery model)
 - Single datacenter deployment (no geo-distribution)
 - Clock drift bounded (no TrueTime required)
-

6. Acceptance Criteria

The system is considered complete when:

1. **Functional Correctness:**
 - Single-shard transactions execute successfully
 - Cross-shard transactions maintain atomicity
 - Failures do not corrupt data or create partial commits
 2. **Fault Tolerance:**
 - System recovers from primary replica failure
 - System recovers from coordinator crash during 2PC
 - WAL replay restores consistent state
 3. **Documentation:**
 - All components have clear documentation
 - UML diagrams (class, sequence, deployment) provided
 - Test cases demonstrate correctness
 4. **Testing:**
 - 70% unit test coverage
 - Integration tests for all transaction types
 - Fault injection tests pass
 - Performance benchmarks meet targets
-

7. Appendices

7.1 Transaction Examples

Example 1: Single-Shard Success

- Transfer \$100 from AccountA to AccountB (both on Shard1)
- Shard1 validates balance, writes to WAL, updates Ledger atomically
- Result: COMMITTED

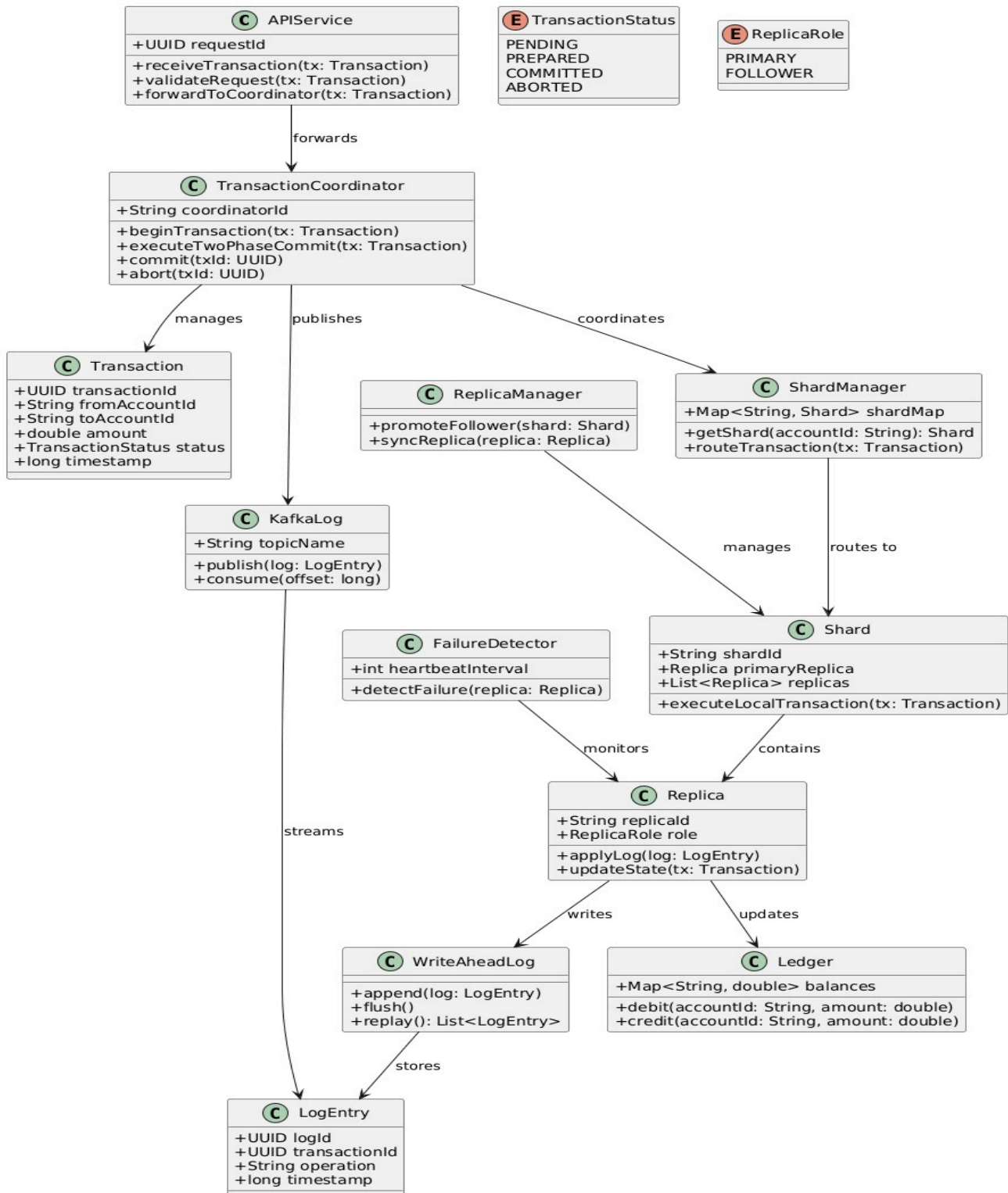
Example 2: Cross-Shard Success

- Transfer \$50 from AccountA (Shard1) to AccountC (Shard2)
- Coordinator executes 2PC: both shards PREPARE → Coordinator issues COMMIT
- Result: COMMITTED

Example 3: Cross-Shard Abort

- Transfer \$1000 from AccountA (Shard1, balance \$100) to AccountC (Shard2)
- Shard1 responds ABORT (insufficient funds)
- Coordinator aborts transaction
- Result: ABORTED

UML Diagrams



System Architecture

